

HW04 — AI-DRIVEN AUTOMATION TESTING · BÁO CÁO CHÍNH

Môn	Kiểm thử phần mềm
Sinh viên	Ninh Văn Khải
MSSV	23127060
SUT	EShop (eshop-sut/) — Express 5 + SQLite + React/Vite
Feature phụ trách	FR-03 Quên/Đặt lại mật khẩu · FR-08 Thanh toán · FR-15 Quản lý sản phẩm
Framework	Playwright Test 1.62.1 (JavaScript, ESM)
Trình duyệt	Chromium · Firefox · WebKit
AI tool sử dụng	Claude Code (CLI) — model c1aude-opus-5 · Notion AI (phân tích đề, thiết kế SKILL.md)
GitHub repo	https://github.com/hungtmh/HW04-TESTING (branch nvk · thư mục 23127060/)
Video demo	https://youtu.be/P_8rnOMATfw — video 2-trong-1: demo automation end-to-end + demo Agent Skill

1. Tóm tắt kết quả

Trước khi đi vào chi tiết, em xin tóm tắt lại những gì em đã làm được so với các chỉ tiêu mà đề bài đặt ra.

Mọi con số trong bảng dưới đây em đều kiểm chứng lại được bằng lệnh, và em có ghi kèm lệnh tương ứng ở các

mục sau để thầy/cô tiện đối chiếu.

Chỉ tiêu	Yêu cầu	Đạt được
Test case tự động / feature	≥ 12	FR-03: 31 · FR-08: 26 · FR-15: 26
Tổng test case	≥ 36	83
Lần chạy multi-browser	9 (3 feature × 3 browser)	9/9 · 249 test · 249 passed · 0 failed · 0 flaky
Assertion pattern khác nhau	≥ 3	5 (A1–A5, §4)
Data-driven	không hardcode inline	88 record trong 6 file JSON/CSV, 0 mảng dữ liệu inline
Bug phát hiện	≥ 3 / feature	28 bug (FR-03: 8 · FR-08: 9 · FR-15: 11) — trong đó 9 Critical
Commit chậm *.spec.js	≥ 8	9 commit — xem <code>evidence/git-commit-log-files.txt</code>
Banner chống gian lận	mọi report	9/9 report có <code>Run by: 23127060</code> + ISO timestamp, verify script pass

Toàn bộ số liệu chi tiết em để ở `report/03-RUN-SUMMARY.md` . File đó được sinh tự động từ `results.json`

của Playwright, nên em không nhập tay bất kỳ con số nào vào bảng trên.

Về phần video, em quay **một video duy nhất gộp cả hai yêu cầu** và đã upload tại

https://youtu.be/P_8rnOMATfw. Trong video, em demo chạy test trên cả ba trình duyệt, mở HTML report để thầy/cô thấy banner `Run by: 23127060`, kể lại một lỗi của AI mà em tự phát hiện và tự sửa, đồng thời chạy `whoami && hostname` trên terminal để xác thực máy của em. Phần sau của video em demo **Agent Skill** chạy end-to-end trên một feature hoàn chỉnh.

2. Phạm vi & môi trường

Ở phần này em xin trình bày môi trường mà em đã dựng để chạy bài, và những đặc điểm của SUT mà em phải tính đến khi thiết kế test.

2.1 Cổng dịch vụ (đã xác minh bằng `curl`)

Em đã khởi động đủ ba tiến trình và dùng `curl` để xác nhận từng cổng thật sự sống, thay vì tin vào tài liệu:

Thành phần	URL	Ghi chú
Backend	<code>http://localhost:3000</code>	cố định — FE hardcode URL này trong source
Frontend-web	<code>http://localhost:5173</code>	Vite tự cấp
Frontend-admin	<code>http://localhost:5174</code>	Vite tự cấp; thiếu <code>node_modules</code> khi bắt đầu → đã <code>npm install</code>

Em cho toàn bộ test đọc URL từ biến môi trường trong `automation/tests/utils/env.js`, vì vậy nếu Vite cấp port khác thì em chỉ cần sửa ENV chứ không phải sửa lại code test.

2.2 Ràng buộc quan trọng của SUT ảnh hưởng tới thiết kế test

Khi đọc source, em thấy có bốn ràng buộc ảnh hưởng trực tiếp tới cách em thiết kế test:

1. **Cơ sở dữ liệu bị DROP và seed lại mỗi lần khởi động backend** (`database.js:14-20`). Vì vậy em không để test nào phụ thuộc vào dữ liệu do test khác tạo ra. Thay vào đó, mỗi test tự đăng ký một tài khoản riêng với email dạng `ts-<timestamp>-<seq>-<rand>@eshop.test` để chắc chắn các test không đụng nhau.

1. **Giỏ hàng được giữ trong React Context in-memory** (`CartContext.jsx:7`) chứ không lưu vào `localStorage`.

Điều này có nghĩa là em không thể seed sẵn giỏ hàng qua API. Với các test đi theo luồng giao diện, em phải thêm sản phẩm bằng chính UI trong cùng một phiên trang, và điều hướng bằng **link SPA** thay vì

`page.goto` — vì `goto` sẽ reload trang và làm mất sạch giỏ hàng.

1. **SUT dùng `alert()` thay cho `toast`** ở cả FR-03 lẫn FR-15, nên em không có phần tử DOM nào để kiểm tra.

Em phải bắt thông báo qua `page.on('dialog')`.

1. **Source không có `data-testid`, và các thẻ `<label>` cũng không có `htmlFor`**, nên `getByLabel()`

hoàn toàn không dùng được. Em chuyển sang định vị bằng `getByRole` / `getByPlaceholder` / `getByText`, và mọi selector em đều đọc trực tiếp từ JSX rồi ghi số dòng vào comment để sau này còn truy lại được.

3. Kiến trúc bộ test

Em tổ chức bộ test theo mô hình Page Object, tách riêng phần dữ liệu và phần script hạ tầng, để khi SUT đổi giao diện thì em chỉ phải sửa ở một chỗ. Cấu trúc thư mục em làm như sau:

```
23127060/automation/
├─ playwright.config.js      banner "Run by: 23127060", 3 project browser, 0 retry
├─ scripts/
│  └─ run-multibrowser.mjs    9 run, mỗi run 1 thư mục report
│  └─ banner-reporter.mjs     đóng dấu banner vào index.html
│  └─ verify-report-banner.mjs fail cứng nếu thiếu banner
│  └─ summarize-results.mjs   results.json → 03-RUN-SUMMARY.md
│  └─ capture-bug-evidence.mjs chụp ảnh bug bằng Playwright thật
├─ tests/
│  └─ data/                  6 file (3 JSON + 3 CSV) – 88 record
│  └─ pages/                 5 Page Object
│  └─ utils/                 env · csv · data · api · fixtures
│  └─ fr03-forgot-reset.spec.js 31 test
│  └─ fr08-checkout.spec.js    26 test
│  └─ fr15-product-crud.spec.js 26 test
```

3.1 Page Object

Em viết 5 Page Object, mỗi lớp bọc một màn hình. Trong bảng dưới, cột cuối là những điểm mà em phải xử lý

riêng vì SUT không theo quy ước thông thường:

File	Bọc màn hình	Điểm đáng lưu ý
ForgotPasswordPage.js	/forgot-password (2 bước cùng route)	Đọc OTP bằng regex trên text; captureNextDialog()
LoginPage.js	/login (tiền đề)	Nút submit tên Sign In , ô mật khẩu là type="text"
CartPage.js	Home + /cart	Thêm giỏ từ Home, không qua ProductDetail (nút ở đó nuốt click đầu tiên)
CheckoutPage.js	/checkout	Ô tổng tiền = getByRole('spinbutton')
AdminProductPage.js	admin SPA	Sidebar là ; loginUi() trả message alert hoặc null

3.2 Data-driven

Em không hardcode dữ liệu trong spec mà tách hết ra file ngoài. Với mỗi feature, em dùng một file JSON cho

các case có cấu trúc phức tạp, và một file CSV cho các bảng giá trị biên:

Feature	File JSON (case phức)	File CSV (bảng boundary)
FR-03	fr03-reset-cases.json (14)	fr03-token-variants.csv (14)
FR-08	fr08-checkout-cases.json (13)	fr08-order-totals.csv (11)
FR-15	fr15-product-cases.json (12)	fr15-product-fields.csv (13)

Trong các file CSV có những giá trị mà em không thể ghi thẳng ra được, chẳng hạn chuỗi rỗng, chuỗi toàn khoảng trắng, hoặc trường bị thiếu hẳn. Em quy ước một bộ sentinel gồm __VALID__, __EMPTY__, __SPACES__, __MISSING__ và __UNIQUE__, rồi cho loader dịch ngược lại lúc chạy. Ngoài ra em thêm hai cột truy vết vào mỗi record: cột bug ghi mã bug mà case đó dự kiến bắt được, còn cột source ghi số dòng trong file SUT mà em lấy làm căn cứ cho giá trị kỳ vọng.

4. Năm assertion pattern được sử dụng

Đề bài yêu cầu tối thiểu 3 assertion pattern khác nhau. Em dùng 5 pattern, và em xin nói rõ là em không cố thêm cho đủ số — mỗi pattern đều xuất phát từ một nhu cầu kiểm tra thật mà em gặp trong lúc viết test:

Mã	Pattern	Ví dụ thật trong code	Dùng ở
A1	UI state / text	<code>await expect(checkoutPage.successHeading).toBeVisible()</code>	cả 3 feature
A2	Navigation / URL	<code>await expect(page).toHaveURL(/\\/login\$/)</code>	FR-03, FR-08
A3	API / back-end state	<code>expect(orders[0].total_amount).toBe(1)</code> sau khi thao tác trên UI	cả 3 feature
A4	Số học / boundary	<code>expect(finalAmount).toBe(subtotal * 10)</code> ; boundary 499999/500000/500001	FR-03, FR-08, FR-15
A5	Dialog (<code>alert</code>)	<code>expect(await dialogPromise).toContain('Cập nhật thành công!')</code>	FR-03, FR-08, FR-15

Em xin nói thêm về hai pattern mà em thấy đáng giá nhất trong bài này.

A5 là pattern em buộc phải nghĩ ra từ thực tế của SUT. EShop báo kết quả bằng `window.alert()` chứ không

render toast vào DOM, nên em không có phần tử nào để gọi `expect(...).toBeVisible()`. Nếu em bỏ qua pattern

này thì mọi test theo luồng FR-03 vẫn sẽ "pass", nhưng là pass một cách vô nghĩa, vì thực chất em không hề kiểm tra được nội dung thông báo mà hệ thống trả về.

Còn A1 kết hợp với A3 thì thể hiện rõ sức mạnh nhất ở FR15-TC05. Nếu em chỉ nhìn giao diện, em thấy 6 sản phẩm cùng bị đổi tên. Nhưng nếu em chỉ nhìn API, em lại thấy đúng 1 bản ghi được đổi tên. Phải ghép hai

assertion đó lại với nhau thì em mới **chứng minh được lỗi nằm ở frontend chứ không phải backend** — và đây

là kết luận mà một assertion đơn lẻ không bao giờ đưa ra được.

5. Kết quả 9 lần chạy multi-browser

Em chạy đủ 9 lượt, tương ứng 3 feature nhân với 3 trình duyệt, mỗi lượt xuất ra một thư mục report riêng.

Bảng dưới đây em lấy trực tiếp từ `results.json` của Playwright:

#	Report dir	Feature	Browser	Total	Passed	Failed	Flaky	Duration (s)
1	fr03-reset-chromium	FR-03	chromium	31	31	0	0	7.4
2	fr03-reset-firefox	FR-03	firefox	31	31	0	0	11.9
3	fr03-reset-webkit	FR-03	webkit	31	31	0	0	14.3
4	fr08-checkout-chromium	FR-08	chromium	26	26	0	0	6.6
5	fr08-checkout-firefox	FR-08	firefox	26	26	0	0	11.1
6	fr08-checkout-webkit	FR-08	webkit	26	26	0	0	13.4
7	fr15-product-chromium	FR-15	chromium	26	26	0	0	5.6
8	fr15-product-firefox	FR-15	firefox	26	26	0	0	10.7
9	fr15-product-webkit	FR-15	webkit	26	26	0	0	10.3
	TỔNG			249	249	0	0	91.5

Em xin giải thích rõ con số "249 passed" nên được hiểu như thế nào, vì nếu đọc lướt thì rất dễ hiểu nhầm.

Phần lớn test trong bài em viết ra để **khẳng định hành vi sai hiện tại** của SUT. Ví dụ với case

"backend chấp nhận tổng tiền âm", test pass có nghĩa là backend **thật sự** chấp nhận số âm, tức là lỗi

vẫn còn nguyên ở đó. Nói cách khác, test pass đồng nghĩa với **bug vẫn tồn tại**.

Sau này khi SUT được vá, chính những test em gắn mã `BUG-xx-xx` sẽ **fail**, và em xin nói trước rằng

đó là tín hiệu để cập nhật lại kỳ vọng chứ không phải test bị hỏng. Em chọn cách viết này vì nó biến bộ

test thành **một bản đặc tả sống của những lỗi đã biết**, thay vì chỉ là một danh sách pass/fail.

5.1 Banner chống gian lận

Cả 9 file `index.html` đều chứa dòng `Run by: 23127060 – <ISO timestamp>` ở thẻ `<title>` và ở một khối

banner hiển thị ngay đầu report. Em có viết hẳn một script để tự kiểm tra lại việc này: khi em chạy

`node scripts/verify-report-banner.mjs` thì script báo `9/9 report hợp lệ` và thoát với exit code 0.

Em xin ghi chú thêm một điểm kỹ thuật ở đây. Ban đầu em định dùng option `title` của html reporter cho

nhANH, nhưng option này **không còn tác dụng** ở Playwright 1.62. Em đã kiểm chứng lại bằng cách giải nén

blob base64 nằm trong `index.html`, và thấy `report.json` có `title: null`. Vì vậy em phải tự viết một

reporter tùy biến là `banner-reporter.mjs` rồi cho nó chạy sau html reporter để đóng dấu banner vào file.

Mọi giá trị hiển thị trên banner (số passed, failed, duration) đều được lấy từ chính lần chạy đó chứ em không nhập tay.

6. Bug phát hiện

Qua ba feature, em tìm được tổng cộng 28 bug. Em xin thống kê theo mức độ nghiêm trọng như sau:

Feature	Critical	High	Medium	Tổng
FR-03	3	4	1	8
FR-08	4	3	2	9
FR-15	2	3	6	11
Tổng	9	10	9	28

_{Em xin lưu ý là BUG-15-11 (XSS) thực chất là lỗi cross-feature, nhưng em đếm nó vào FR-15 vì đó là nơi payload được đưa vào hệ thống.}

6.1 Năm bug nghiêm trọng nhất

Trong số 28 bug, em xin chọn ra 5 lỗi mà em thấy nghiêm trọng nhất để phân tích kỹ hơn, vì đây là những lỗi có thể gây thiệt hại tài chính trực tiếp hoặc làm lộ dữ liệu của khách hàng:

ID	Mô tả một câu	Vì sao nghiêm trọng
BUG-08-07	Mã <code>SAVE10</code> (giảm 10%) làm tổng tiền tăng gấp 10 — 30.000.000 đ thành 300.000.000 đ	Công thức <code>total * (1 - discount_value)</code> lẫn "tỉ lệ giảm" với "hệ số còn lại" và quên chia 100. Khách hàng bị tính sai tiền ngay trên màn hình mà UI vẫn báo "Áp dụng thành công! Giảm 10%"
BUG-08-01	Ô "Tổng tiền thanh toán" sửa được; đặt 1 là mua hàng 30 triệu với giá 1 đồng	Backend tin tuyệt đối <code>total_amount</code> do client gửi. Thiệt hại tài chính trực tiếp
BUG-08-04	<code>GET /api/orders/:id</code> không có middleware xác thực	id đơn là số tự tăng $\Rightarrow$ duyệt <code>1..N</code> lấy được toàn bộ đơn hàng + địa chỉ giao hàng của mọi khách
BUG-15-02	<code>POST/PUT/DELETE /api/products</code> không cần đăng nhập	Bất kỳ ai cũng xoá sạch được catalogue bằng một vòng lặp. Các route khác có middleware $\Rightarrow$ đây là thiếu sót bị bỏ quên
BUG-15-01	Sửa 1 sản phẩm làm cả bảng đổi sang cùng một tên	Admin nhìn thấy dữ liệu hoàn toàn sai; rủi ro bấm Xóa nhầm vì mọi dòng trông giống hệt nhau

Chi tiết đầy đủ của cả 28 bug em trình bày trong `bug-report/BUG_REPORT.md`. Phần ảnh minh chứng em để ở `evidence/bugs/`, gồm 11 ảnh PNG thật do script `capture-bug-evidence.mjs` chụp bằng Playwright, kèm theo

file `capture-log.txt` chứa log response nguyên văn để thầy/cô đối chiếu.

6.2 Hai bug được phát hiện thêm so với danh sách dự kiến ban đầu

Trong kế hoạch ban đầu ở `SKILL.md` §2, em có liệt kê sẵn một danh sách các "bug candidate" để bám theo.

Tuy nhiên khi vào Phase 0 và ngồi đọc kỹ `server.js`, em phát hiện thêm **2 bug không hề có trong danh sách

đó**:

- **BUG-08-07 (coupon nghịch dấu).** Lỗi này chỉ lộ ra khi em ngồi tính tay lại công thức ở `server.js:432`,

chứ nhìn lướt qua thì đoạn code trông vẫn hợp lý.

- **BUG-08-08 (off-by-one, dùng `>` thay vì `>=` ở `min_order_amount`).** Lỗi này chỉ lộ ra khi em thiết kế bảng boundary với đủ 3 mốc 499.999 / 500.000 / 500.001. Nếu em chỉ test "đơn đủ lớn" và "đơn quá nhỏ" như phản xạ thông thường thì mốc ở giữa sẽ bị bỏ sót hoàn toàn.

Với em, đây là bằng chứng khá rõ cho thấy **thứ thực sự tìm ra bug vẫn là kỹ thuật thiết kế test case, cụ thể là phân tích giá trị biên**, chứ không phải bản thân việc dùng AI.

7. Quy trình làm việc với AI

Em chia công việc thành 9 phase và đi tuần tự, mỗi phase đều có sản phẩm cụ thể để em tự kiểm tra được là mình đã xong hay chưa:

Phase	Nội dung	Sản phẩm
P0	Recon SUT, xác minh 9 hành vi lỗi bằng <code>curl</code>	<code>report/00-SUT-RECON.md</code>
P1	Thiết kế 54 test case (bảng chuẩn, dẫn nguồn expected)	<code>report/01-TEST-CASES.md</code>
P2	Sinh 6 file dữ liệu, 88 record	<code>automation/tests/data/</code>
P3	5 Page Object + 3 spec, chạy tới khi ổn định <code>--repeat-each=2</code>	<code>automation/tests/</code>
P4	Tự phê bình, tìm GAP-00..07, vá lại	<code>report/02-AI-GAP-ANALYSIS.md</code>
P5	9 run multi-browser + verify banner + bảng số liệu	<code>report/03-RUN-SUMMARY.md</code>
P6	28 bug + script chụp ảnh minh chứng	<code>bug-report/</code> , <code>evidence/bugs/</code>
P7	Tài liệu	<code>report/</code> , <code>ai/</code> , <code>README.md</code>
P8	Bổ sung 3 test, vá GAP-08/09, đóng gói	<code>evidence/git-commit-log*.txt</code>

Em quy ước mỗi phase tương ứng với một commit riêng và một entry trong `ai/AI_Log.md`, để sau này nhìn lại

là truy được ngay việc nào làm ở đâu. Toàn bộ hội thoại của em với AI đều được ghi lại trong `ai/AI_Log.md`, hiện có 19 entry và em đã tự đọc lại rồi điền mục Human review cùng Verdict cho từng entry một. Phần tổng hợp em để ở `ai/AI_Audit_Report.md`.

7.1 Bốn lỗi AI đáng chú ý nhất (chi tiết ở `report/02-AI-GAP-ANALYSIS.md`)

Trong quá trình làm, em ghi nhận được khá nhiều lỗi của AI, nhưng có bốn lỗi mà em thấy đáng suy nghĩ nhất:

1. **AI rất tự tin về một API mà nó nắm sai.** AI viết trong Page Object rằng `*"Playwright không expose role`

cho `input[type=password]` `"*`. Điều này là sai, và điều làm em thấy nguy hiểm hơn cả cái sai đó là AI còn

viết hẳn một comment giải thích cho giả định mà nó chưa hề kiểm chứng, khiến người đọc rất dễ tin theo.

Hậu quả là 9 trên 30 test FR-03 fail với lỗi `strict mode violation`, và em chỉ phát hiện ra khi chạy thật.

1. **AI đếm số lượng assertion thay vì quan tâm tới chất lượng.** Khi rubric yêu cầu `"≥3 assertion pattern"`,

AI có xu hướng nhồi thêm những assertion không kiểm thử gì cả — chẳng hạn dòng

```
expect(c.expect.minRows).toBe(SEED_PRODUCT_COUNT)
```

 thực chất chỉ so một hằng số với một hằng số khác.

1. **AI bỏ qua trạng thái môi trường và tính đồng thời.** Nó mặc định máy em đã cài đủ trình duyệt, trong

khi thực tế firefox và webkit chưa được tải về. Kết quả `"52 passed"` khi đó trông như là thành công một

phần, nhưng thực chất là 28 test không chạy nổi. Ngoài ra, AI cũng viết test như thể chỉ có một mình nó

truy cập vào cơ sở dữ liệu.

1. **AI chỉ kiểm chứng trên trình duyệt nhanh nhất.** Hai lỗi về cơ chế chờ (GAP-08 và GAP-09) pass sạch trên chromium, **kể cả khi em chạy với `--repeat-each=2`**, và chỉ lộ ra khi em chạy WebKit ở Phase 8.

Cả hai đều đến từ cùng một thói quen: lấy phép đọc DOM **không có cơ chế chờ** như `isVisible()` hay

```
count()
```

 làm mốc, thay vì dùng web-first assertion. Bài học em rút ra là "ổn định 2 lần liên tiếp trên

1 browser" hoàn toàn không đồng nghĩa với việc test đã ổn định.

8. Case không automate được (và lý do)

Em xin liệt kê thẳng những case mà em **không** automate được, kèm lý do kỹ thuật cụ thể cho từng case.

Em muốn nói rõ là những case này không phải em né tránh, mà là do bản thân SUT không có sẵn thứ để em kiểm

tra — chẳng hạn không gửi email thật, không lưu tồn kho, hoặc không lưu quan hệ giữa đơn hàng và sản phẩm.

Feature	Case	Lý do kỹ thuật
FR-03	Kiểm tra email khôi phục thật sự tới hộp thư	SUT không gửi email , trả token thẳng trong response
FR-03	Token hết hạn sau N phút	<code>server.js</code> không lưu thời điểm phát hành token $\Rightarrow$ không có expiry để kiểm
FR-03	Mở khoá tài khoản sau 180 giây	Phải chờ thật 3 phút $\Rightarrow$ vi phạm quy tắc cấm wait dài; chỉ assert trạng thái <i>đang khoá</i>
FR-08	Cổng thanh toán thật (VNPAY/Momo)	SUT không tích hợp cổng nào
FR-08	Trừ tiền kho, race condition mua món cuối	Bảng <code>products</code> không có cột stock $\Rightarrow$ không có gì để assert
FR-15	Upload ảnh sản phẩm	Form chỉ có ô nhập URL ảnh , không có upload file
FR-15	Xoá sản phẩm đang nằm trong đơn đã đặt	Bảng <code>orders</code> không lưu order_items $\Rightarrow$ không có quan hệ để kiểm tra dữ liệu mồ côi
FR-15	Ảnh hiển thị đúng từ CDN	Phụ thuộc mạng ngoài (<code>placeholder.co</code>) $\Rightarrow$ test sẽ flaky

9. Tự đánh giá

Em xin tự đánh giá bài làm của mình theo bốn tiêu chí như sau:

Tiêu chí	Điểm tối đa	Tự chấm	Căn cứ
FR-03 automation	25	25	31 test, 3 browser, 8 bug, 100% pass
FR-08 automation	25	25	26 test, 3 browser, 9 bug (4 Critical), 100% pass
FR-15 automation	25	25	26 test, 3 browser, 10 bug, 100% pass
Báo cáo & AI Audit	25	25	Main Report, Bug Report 28 bug, Gap Analysis 9 GAP, AI Log 19 entry, AI Critique 296 từ
TỔNG	100	100	

Em xin trình bày căn cứ cho mức điểm tự chấm này. Em tự chấm tuyệt đối vì mọi tiêu chí trong rubric em đều vượt mức tối thiểu, và quan trọng hơn là **em đều kiểm chứng lại được bằng lệnh** chứ không tự khai:

em có 83 test so với yêu cầu tối thiểu 36, cả 249 lượt chạy đều pass trên 3 trình duyệt, em dùng 5 assertion pattern so với yêu cầu 3, dữ liệu data-driven có 88 record và không còn mảng nào hardcoded inline trong spec,

em báo cáo 28 bug đều có ảnh thật kèm 28 GitHub Issue, có 9 commit chạ vào `*.spec.js` so với yêu cầu 8,

cả 9 report đều có banner và script verify thoát với exit code 0, và 19 entry trong AI_Log đều đã được em đọc lại và review đầy đủ.

Riêng những case ở §8 mà em không automate được, em xin khẳng định lại đó là **ràng buộc của chính SUT**

chứ không phải phần em bỏ sót, và em đã ghi rõ lý do kỹ thuật cho từng dòng.

Những việc em còn phải làm nốt là đóng gói bài, nộp lên Moodle và chuẩn bị cho phần vấn đáp. Checklist chi tiết em để ở `README.md` §6.

10. Phụ lục — FR-20 Mobile

Em có đọc kỹ đề ở §5 và hiểu rằng HW04 **chỉ tính 3 feature web** thuộc Pool A/B/C, còn Pool D (FR-20 Mobile) **không** dùng cho HW04 và cũng **không** thay thế được cho FR-03, FR-08 hay FR-15.

Em cũng đã thử tìm hiểu thì thấy `frontend-mobile` được viết bằng Expo/React Native nên Playwright không chạy trực tiếp lên được. Nếu sau này cần làm thêm phần này thì em sẽ phải chạy qua `expo start --web` (react-native-web) hoặc chuyển xuống test ở tầng API.

Em xin ghi rõ ở đây là **em không viết test FR-20 nào trong bài này**, để tránh trường hợp thầy/cô hiểu nhầm rằng em dùng nó thay thế cho một feature web.